

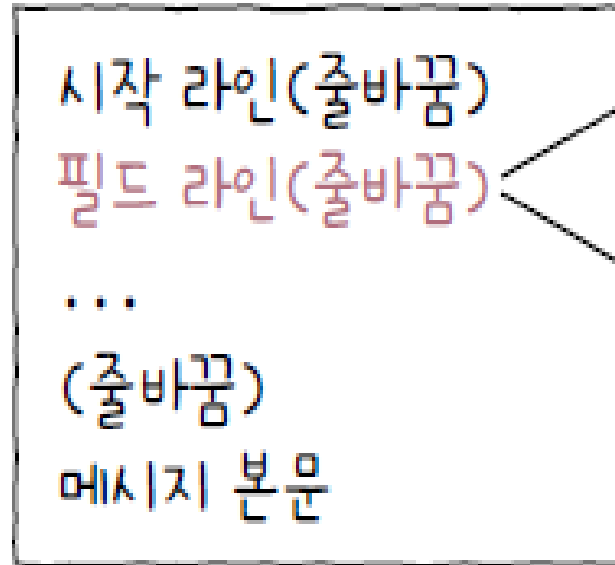


HTTP 헤더와 HTTP 기반 기술

- HTTP의 주요 헤더들
- HTTP 기반 기술인 캐시, 쿠키, 콘텐츠 협상의 개념 및 관련 헤더

HTTP 헤더

- HTTP 헤더는 필드 이름(헤더 이름)과 필드 값(헤더 값)이 콜론(:)을 기준으로 구분
 - 특별한 사전 지식이 필요하지 않은 헤더
 - 사전 지식이 필요한 헤더
 - 예) 캐시, 쿠키, 콘텐츠 협상 관련 헤더
- HTTP 요청 시 주로 사용되는 헤더
- HTTP 응답 시 주로 사용되는 헤더
- HTTP 요청과 응답 모두에서 자주 활용되는 헤더



HTTP 헤더

요청 시 활용되는 HTTP 헤더

① Host

- 요청을 보낼 호스트를 나타내는 헤더
- 주로 도메인 네임으로 명시되며, 포트 번호가 포함되어 있을 수 있음
- 예) `http://info.cern.ch/hypertext/WWW/TheProject.html`에 접속할 때의 HTTP 요청 메시지 일부

```
GET /hypertext/WWW/TheProject.html HTTP/1.1
```

```
Host: info.cern.ch
```

```
...
```

HTTP 헤더와 HTTP 기반 기술(4)

② User-Agent

- 유저 에이전트(user agent) – 웹 브라우저와 같이 HTTP 요청을 시작하는 클라이언트 측의 프로그램
- User-Agent 헤더에는 요청 메시지 생성에 참여한 클라이언트 프로그램과 관련된 다양한 정보가 명시
 - 운영체제, 브라우저 종류 및 버전, 렌더링 엔진 등

웹 브라우저의 Mozilla 호환 여부*

렌더링 엔진** 관련 정보

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/109.0

운영체제 및 아키텍처 정보

브라우저와 버전 정보

* 오늘날 웹 브라우저를 통한 요청 메시지에 대부분 포함된 정보

** 브라우저에 시각적 요소를 구현하는 구성 요소

HTTP 헤더

③ Referer

- Referer는 개발 시 아주 유용한 헤더 중 하나
- 클라이언트가 요청을 보낼 때 머무르고 있던 URL이 명시
- 예) 클라이언트가 `https://en.wikipedia.org`에서 요청을 보냈음을 의미하는 헤더
 - Referer를 통해 클라이언트의 유입 경로를 파악

```
Referer: https://en.wikipedia.org/
```

[참고]

영문법적으로는 Referrer가 맞지만, 초기 개발 당시의 오타로 인해 Referer라는 표기가 오늘날까지 사용

HTTP 헤더

```
Authorization: <type> <credentials>
```

④ Authorization

- 클라이언트의 인증 정보를 담는 헤더
- 인증 타입(type)과 인증을 위한 정보(credentials)가 차례로 명시
- 인증 타입에 따라 인증 정보에 명시될 값이 달라짐
 - 가장 기본적인 HTTP 인증 타입은 Basic
 - username:password와 같이 사용자 아이디(username)와 비밀번호(password)를 콜론을 이용해 합친 뒤,
이를 Base64 인코딩한 값을 인증 정보(credential)로 삼는 방식
 - 예) 사용자 아이디가 'minchul'이고 비밀번호는 '1234'라고 가정
'minchul:1234'를 Base64 방식으로 인코딩하면 'bWluY2h1bDoxMjM0'

```
Authorization: Basic bWluY2h1bDoxMjM0
```

HTTP 헤더

응답 시 활용되는 HTTP 헤더

① Server

- 요청을 처리하는 서버 측의 소프트웨어와 관련된 정보를 명시
- 예) 'Unix 운영체제에서 동작하는 아파치 HTTP 서버'를 의미하는 헤더

```
Server: Apache/2.4.1 (Unix)
```

HTTP 헤더

② Allow

- Allow 헤더는 클라이언트에게 허용된 HTTP 메서드 목록을 알려 주기 위해 사용
- 상태 코드 405(Method Not Allowed)를 응답하는 메시지에서 Allow 헤더가 함께 사용

Request URL:	http://127.0.0.1:8000/users/api/token/
Request Method:	GET
Status Code:	● 405 Method Not Allowed
Remote Address:	127.0.0.1:8000
Referrer Policy:	strict-origin-when-cross-origin
▼ Response Headers ☒ Raw	
HTTP/1.1 405 Method Not Allowed	
Date: Thu, 19 Oct 2023 03:39:28 GMT	
Server: WSGIServer/0.2 CPython/3.10.6	
Content-Type: text/html; charset=utf-8	
Vary: Accept, Cookie	
Allow: POST, OPTIONS	

HTTP 헤더

③ Retry-After

- 상태 코드 503(Service Unavailable) ‘현재는 요청을 처리할 수 없으나 추후 가능할 수도 있음’ 응답과 함께 사용될 수 있는 헤더
- 자원을 사용할 수 있는 날짜 혹은 시각을 나타냄
- 예) ‘2024년 8월 23일 금요일 09시 이후에 사용 가능함’
‘120초 이후에 사용 가능함’

```
Retry-After: Fri, 23 Aug 2024 09:00:00 GMT
```

```
Retry-After: 120
```

HTTP 헤더

④ Location

- 클라이언트에게 자원의 위치를 알려 주기 위해 사용되는 헤더
- 주로 리다이렉션이 발생했을 때나 새로운 자원이 생성되었을 때 사용

⑤ WWW-Authenticate

- 상태 코드 401(Unauthorized)과 함께 사용되는 헤더
- 자원에 접근하기 위한 인증 방식을 설명

```
WWW-Authenticate: Basic
```

- 실제로는 이보다 조금 더 많은 정보를 알려 주는 경우가 많음
 - 예) 보안 영역(realm), 인증에 사용될 문자집합(charset) 등

```
WWW-Authenticate: Basic realm="Access to engineering site", charset="UTF-8"
```

HTTP 헤더



영역(realm)이란?

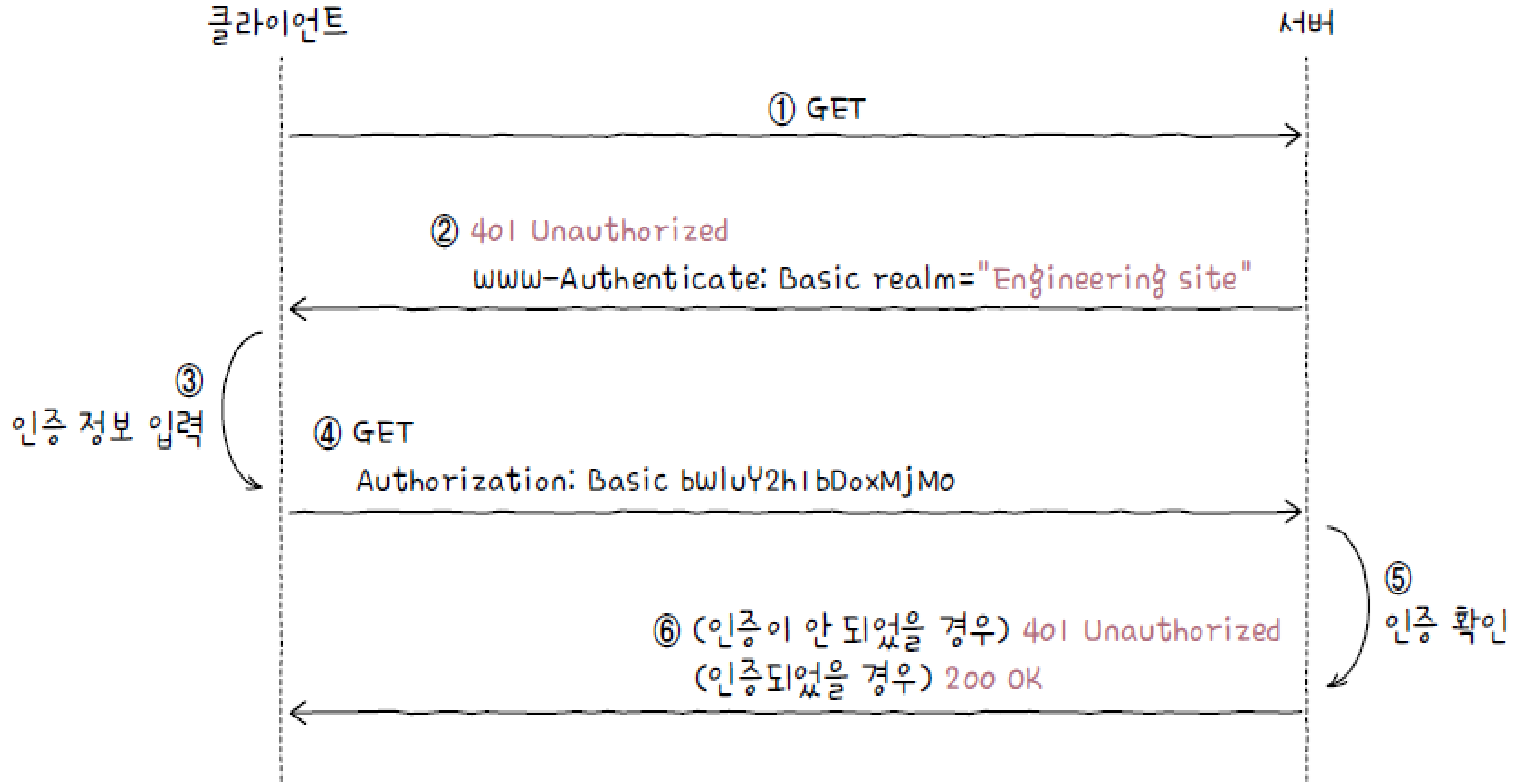
- WWW-Authenticate 헤더에서 'realm'은 보안이 적용될 영역을 의미
- 영역이 달라지면 요구되는 권한도 달라질 수 있음
 - 같은 서버가 제공하는 자원일지라도 'Engineering site'라는 영역에 속한 자원에 접근 가능한 사용자는 'Financial site'라는 영역에 속한 자원에 접근이 불가능할 수 있음

HTTP 헤더

- HTTP 인증(Basic 인증) 수행 과정

- Authorization과 WWW-Authenticate 헤더를 통해 인증되지 않은 클라이언트가 HTTP 인증(Basic 인증)을 수행하는 과정
 1. 인증되지 않은 클라이언트가 서버에 GET 요청 메시지를 전송
 2. 서버는 클라이언트에게 상태 코드 401(Unauthorized)과 함께 WWW-Authenticate 헤더를 통해 인증 방식을 알림
 3. 클라이언트는 사용자로부터 인증 정보(사용자 아이디와 비밀번호)를 전달받음
 4. “사용자 아이디:비밀번호”를 Base64 인코딩한 값을 인증 정보로 삼은 Authorization 헤더를 통해 다시 GET 요청 메시지를 전송
 5. 서버는 인증 정보를 확인
 6. 인증이 유효하면 상태 코드 200으로 응답하고, 인증되지 않았으면 상태 코드 401로 응답

HTTP 헤더



HTTP 헤더

요청과 응답 모두에서 활용되는 HTTP 헤더

① Date

- 메시지가 생성된 날짜와 시각에 관련된 정보를 담은 헤더
- 클라이언트와 서버 모두에서 사용될 수 있는 헤더

```
Date: Tue, 15 Nov 1994 08:12:31 GMT
```

② Connection

- 클라이언트의 요청과 응답 간의 연결 방식을 설정하는 헤더
- 지속 연결이 Connection에 명시되는 대표적인 연결 방식
- Connection 필드에는 다양한 값이 명시될 수 있지만, 가장 대표적으로 사용되는 값은 keep-alive와 close

```
Connection: keep-alive  
Connection: close
```

HTTP 헤더

③ Content-Length

- 본문의 바이트 단위 크기(길이)

```
Content-Length: 100
```

④ Content-Type, Content-Language, Content-Encoding(1)

- 메시지 본문의 표현 방식을 설명하는 헤더
- 표현 헤더(representation header)의 일종

Content-Type 헤더

- 메시지 본문에서 사용된 미디어 타입

```
Content-Type: text/html; charset=UTF-8
```

HTTP 헤더

④ Content-Type, Content-Language, Content-Encoding(2)

Content-Language 헤더

- 메시지 본문에 사용된 자연어를 명시
- 언어 태그로 명시되며, 언어 태그는 하이픈(-)으로 구분된 구조
 - 예) Content-Language 헤더 값이 en이거나 ko일 경우 첫 번째 서브 태그만 사용
Content-Language 헤더 값이 en-US, ko-KR일 경우 두 번째 서브 태그까지 사용

<첫 번째 서브 태그>

<첫 번째 서브 태그>-<두 번째 서브 태그>

<첫 번째 서브 태그>-<두 번째 서브 태그>-<세 번째 서브 태그>

...

언어	언어 코드
한국어	ko
영어	en
중국어	zh
일본어	ja
독일어	de
프랑스어	fr

국가	국가 코드
한국	KR
미국	US
영국	GB
중국	CN
타이완	TW
일본	JP
독일	DE
프랑스	FR

HTTP 헤더

④ Content-Type, Content-Language, Content-Encoding(3)

Content-Encoding 헤더

- 메시지 본문을 압축하거나 변환한 방식이 명시
- 헤더에 명시될 수 있는 대표적인 값은 'gzip', 'compress', 'deflate', 'br' 등

```
Content-Encoding: gzip
```

```
Content-Encoding: compress
```

```
Content-Encoding: deflate
```

```
Content-Encoding: br
```

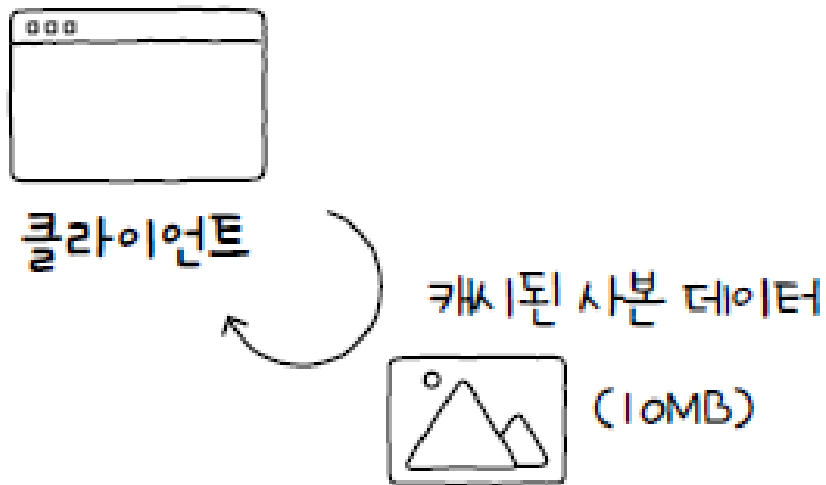
```
// 여러 인코딩이 사용되었을 경우: 적용된 순서대로 명시
```

```
Content-Encoding: deflate, gzip
```

HTTP 기반 기술

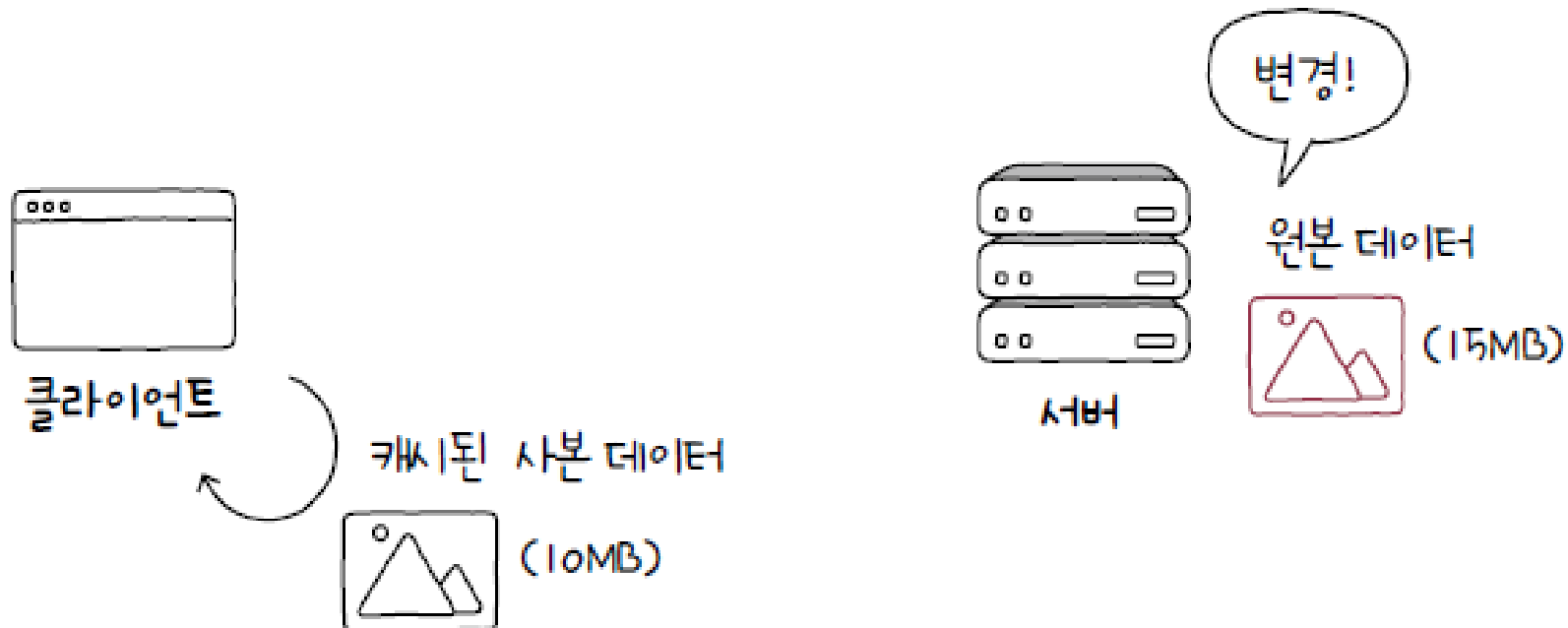
캐시(cache)

- 불필요한 대역폭 낭비와 응답 지연을 방지하기 위해 정보의 사본을 임시로 저장하는 기술
 - 사본을 임시로 저장해 두면 동일한 요청에 대해 캐시된 데이터를 활용할 수 있기 때문에 불필요한 대역폭 낭비를 줄일 수 있고, 더 빠르게 데이터에 접근 가능
 - 개인 전용 캐시(private cache) : 웹 브라우저에 저장
 - 공용 캐시(public cache) : 클라이언트와 서버 사이에 위치한 중간 서버에 저장



HTTP 기반 기술

- 캐시 신선도의 검사와 유지
 - 캐시 신선도(cache freshness)
 - 캐시된 사본 데이터가 얼마나 최신 원본 데이터와 유사한지를 표현



HTTP 기반 기술

- 캐시 신선도의 검사

- ‘캐시된 데이터에 유효 기간을 설정
 - 캐시 데이터에 유효 기간을 설정하고, 기간이 만료되었다면 원본 데이터를 다시 요청하는 방식
- 캐시할 데이터에 유효 기간을 부여
 - 응답 메시지의 Expires 헤더(날짜)와 Cache-Control 헤더의 Max-Age 값(초)을 사용
 - 예) 캐시의 유효 시간을 2024년 2월 6일 화요일 12:00:00로 설정, 또는 1200초로 설정하는 응답 메시지

응답 메시지

```
HTTP/1.1 200 OK
Date: Mon, 05 Feb 2024 12:00:00 GMT
Content-type: text/plain
Content-length: 100
Expires: Tue, 06 Feb 2024 12:00:00 GMT
...본문 생략...
```

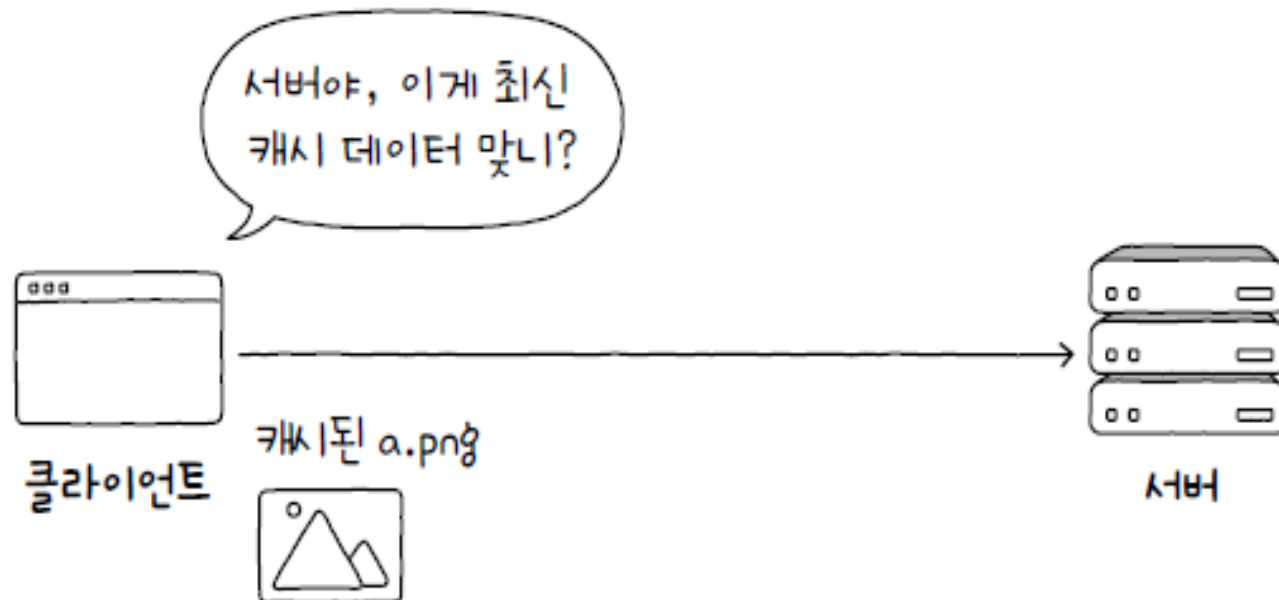
```
HTTP/1.1 200 OK
Date: Mon, 05 Feb 2024 12:00:00 GMT
Content-type: text/plain
Content-length: 100
Cache-Control: max-age=1200
...본문 생략...
```

HTTP 기반 기술

- 캐시 신선도의 재검사(1)

1) 날짜를 기반으로 서버에게 물어보는 방법

- 클라이언트는 If-Modified-Since 헤더를 통해 서버에게 특정 시점 이후로 원본 데이터에 변경이 있었는지 물어볼 수 있음
- If-Modified-Since 헤더의 값으로 특정 시점(날짜와 시각)이 명시되는데, 이 시점 이후로 원본에 변경이 있었다면 그때만 새 자원으로 응답하도록 서버에게 요청하는 헤더



HTTP 기반 기술

- 예) '2024년 8월 23일 금요일 09:00:00 이후에 `www.example.com/index.html`의 자원이 변경되었는가?
 - 변경이 되었을 경우에만 새 자원으로 응답하라는 요청 메시지와 서버 응답

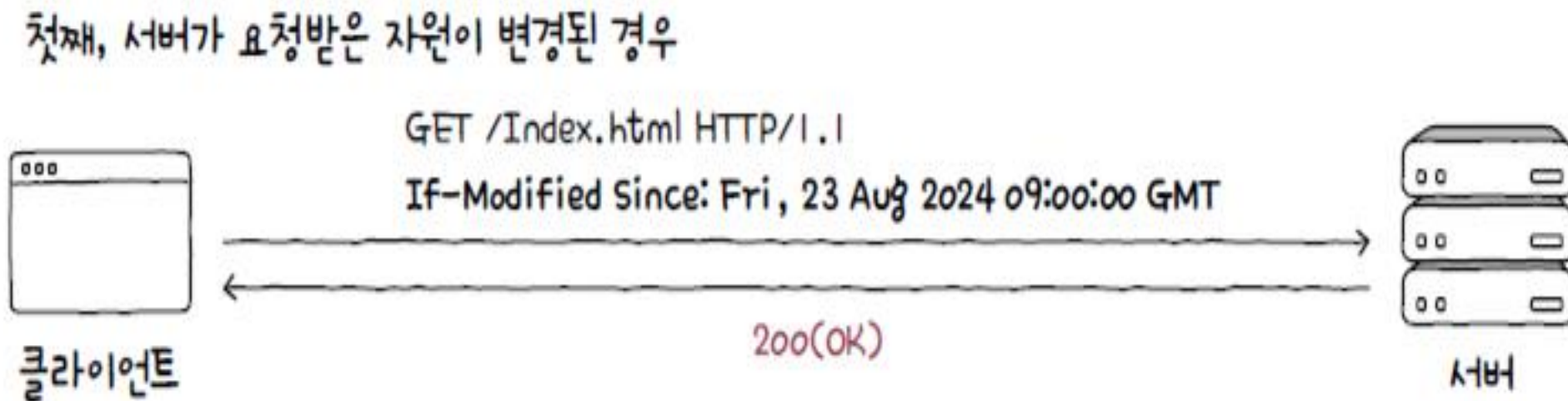
요청 메시지

```
GET /index.html HTTP/1.1
Host: www.example.com
If-Modified-Since: Fri, 23 Aug 2024 09:00:00 GMT
```

1. 요청받은 자원이 변경되었음
2. 요청받은 자원이 변경되지 않았음
3. 요청받은 자원이 삭제되었음

HTTP 기반 기술

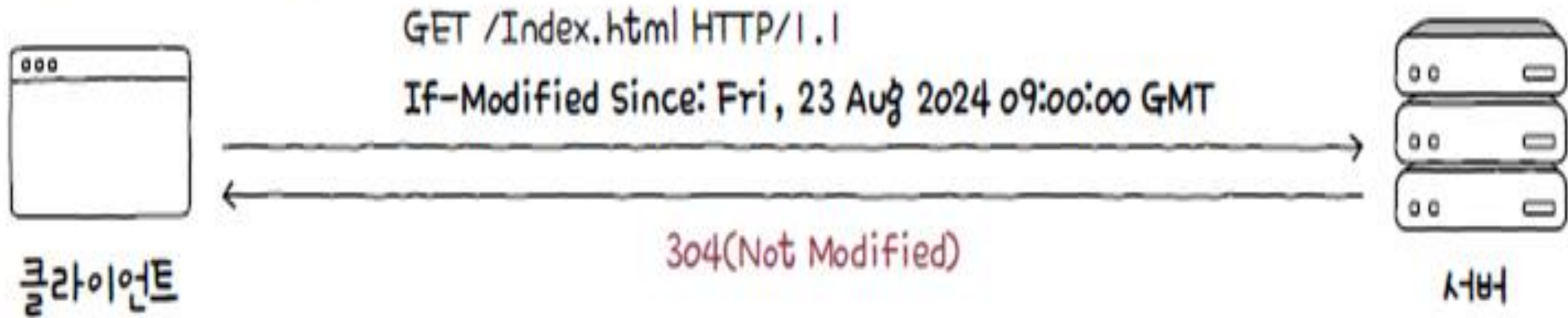
① 요청받은 자원이 변경되었음



HTTP 기반 기술

② 요청받은 자원이 변경되지 않았음

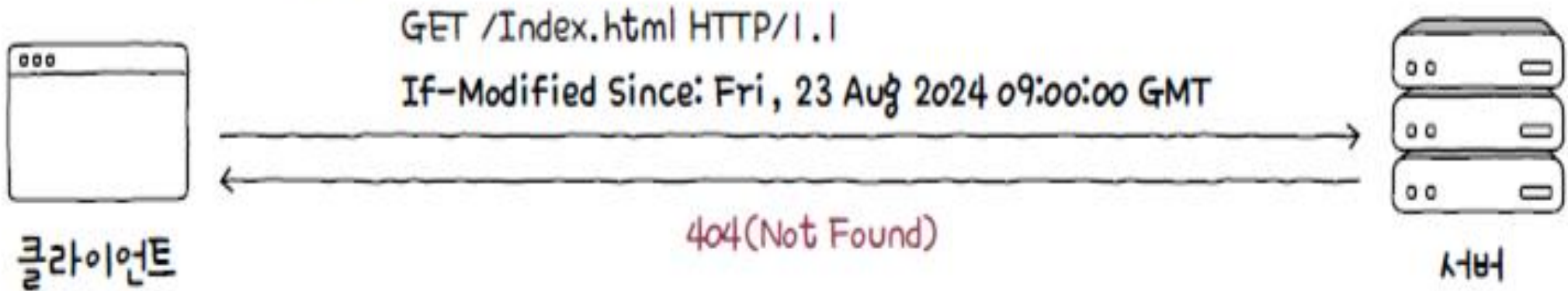
둘째, 서버가 요청받은 자원이 변경되지 않은 경우



HTTP 기반 기술

③ 요청받은 자원이 삭제되었음

셋째, 서버가 요청받은 자원이 삭제된 경우



HTTP 기반 기술

- Last-Modified 헤더

- 서버는 상태 코드 304(Not Modified)를 통한 자원의 '변경 여부'뿐만 아니라 자원의 '마지막 변경 시점'도 클라이언트에게 알려 줄 수 있음

- 특정 자원이 마지막으로 수정된 시점

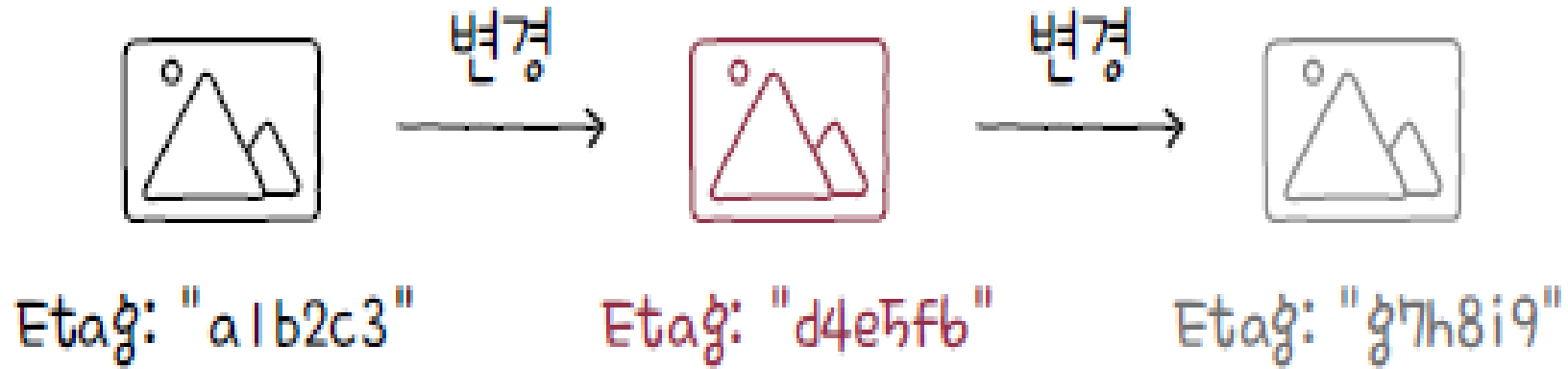
2019년 10월 17일 목요일 7시 18분 26초에 변경되었음

Name	X	Headers	Preview	Response	Initiator	Timing
example.com						
		▼ General				
		Request URL:		http://example.com/		
		Request Method:		GET		
		Status Code:		● 304 Not Modified		
		Remote Address:		93.184.216.34:80		
		Referrer Policy:		strict-origin-when-cross-origin		
		▼ Response Headers				
				☒ Raw		
				HTTP/1.1 304 Not Modified		
				Age: 462518		
				Cache-Control: max-age=604800		
				Date: Mon, 16 Oct 2023 10:00:02 GMT		
				Etag: "3147526947+gzip"		
				Expires: Mon, 23 Oct 2023 10:00:02 GMT		
				Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT		

HTTP 기반 기술

2) 엔티티 태그를 기반으로 서버에게 물어보는 방법

- Etag는 '자원의 버전'을 식별하기 위한 정보
 - 버전 version이란 '유의미한 변경 사항'을 의미
 - 즉, 자원이 변경될 때마다 자원의 버전을 식별하는 Etag 값이 변경
 - 반대로 자원이 변경되지 않았다면 Etag 값도 변경되지 않음



HTTP 기반 기술

- If-None-Match 헤더

- 클라이언트가 Etag 값이 부여된 자원을 캐시할 때 캐시 신선도를 검사하기 위해 서버에게 ‘이 Etag 값과 일치하는 자원이 있는가?’
- 예) ‘혹시 Etag 값이 abc인 www.example.com/index.html이라는 자원이 있는가?’
 - 자원이 변경되었다면(Etag 값이 바뀌었다면) 그때만 새 자원으로 응답해 줘’라는 요청 메시지와 응답

```
GET /index.html HTTP/1.1
Host: www.example.com
If-None-Match: "abc"
```

1. 요청받은 자원이 변경되었음(Etag 값이 변경됨)
 - 서버는 상태 코드 200(OK)과 함께 변경된 데이터와 Etag 값을 응답
2. 요청받은 자원이 변경되지 않았음(Etag 값이 동일함)
 - 서버는 메시지 본문 없는 상태 코드 304(Not Modified)를 응답
3. 요청받은 자원이 삭제되었음
 - 상태 코드 404(Not Found)를 응답

HTTP 기반 기술

쿠키(cookie)

- 서버에서 생성되어 클라이언트 측에 저장되는 데이터
 - 상태를 유지하지 않는 HTTP의 특성을 보완하기 위한 수단
 - 쿠키를 이루는 정보는 기본적으로 <이름, 값> 쌍의 형태를 띠고 있고, 추가로 적용 범위와 만료 기간 등 다양한 속성을 가질 수 있음
- 서버는 쿠키를 생성하여 클라이언트에게 전송하고, 클라이언트는 전달받은 쿠키를 저장해 두었다가 추후 동일한 서버에 보내는 요청 메시지에 쿠키를 포함하여 전송
- 서버는 쿠키 정보를 참고해 두 개의 요청이 같은 클라이언트에서 왔는지, 로그인 상태를 유지하고 있는지 등을 알 수 있음

HTTP 기반기술

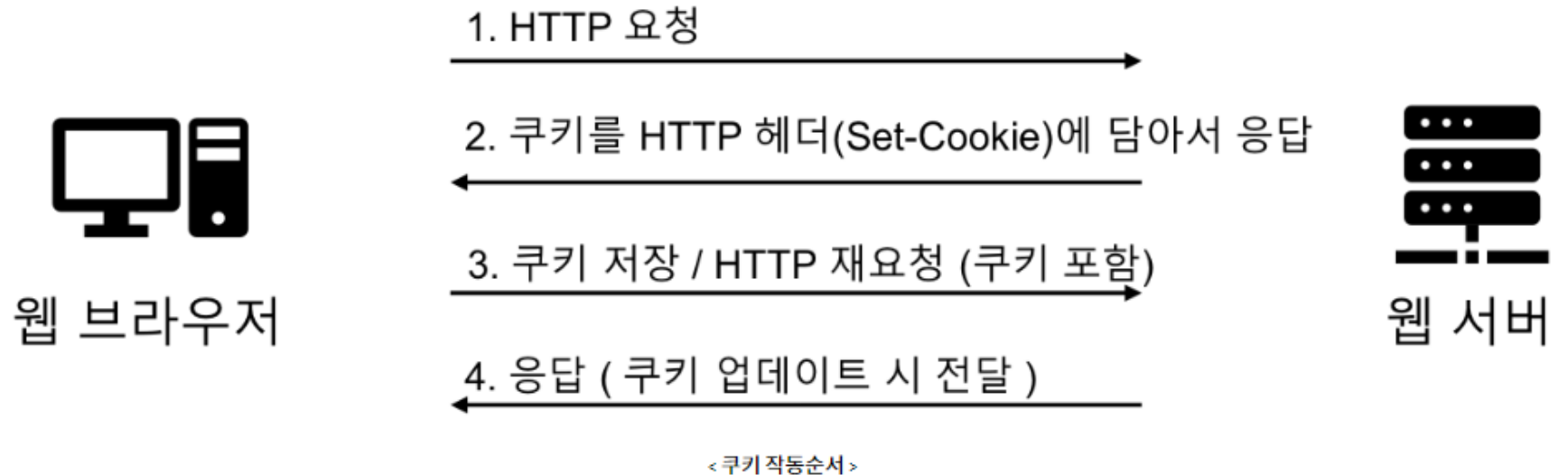
- 쿠키 사용 목적

세션 관리: 로그인, 장바구니, 게임스코어 등 정보관리

개인화: 사용자 선호, 테마 세팅

트래킹: 사용자 행동 기록 분석

- HTTP는 서버에 클라이언트의 상태를 저장하지 않는 stateless 특성을 갖기 때문



- 모든 요청마다 쿠키가 함께 전송되기 때문에, 특히 mobile data connections에서 성능이 떨어지는 원인이 되어 modern storage APIs 권장

- **HTTP는 스테이트리스 프로토콜**

- 같은 클라이언트가 서버에 여러 번 요청을 보낸다고 해도, 기본적으로 서버는 모든 요청들을 별개의 요청으로 간주
- 클라이언트가 서버에 요청 메시지를 보낼 때마다 (아이디, 비밀번호와 같은) 인증 정보를 보내고 번거로운 인증 과정을 거쳐야 하는 것일까?

- **세션 아이디(session id)와 세션 인증(session authentication)**

1. 클라이언트는 서버에게 (아이디, 비밀번호와 같은) 인증 정보를 전송
 2. 인증 정보가 올바르다면, 서버는 세션 아이디를 생성해 클라이언트에게 전송
 3. 서버는 생성한 세션 아이디를 데이터베이스 등에 저장
 4. 클라이언트는 추후 요청을 보낼 때 쿠키 내에 세션 아이디를 포함하여 전송
 5. 서버는 쿠키 속 세션 아이디와 저장된 세션 아이디를 비교하여 클라이언트를 식별
- 위와 같이 쿠키를 통해 세션 아이디를 전송하면 요청을 보낼 때마다 번거로운 인증 과정을 거칠 필요가 없어서 효율적

HTTP 기반 기술

- 쿠키는 서버가 생성하고, 클라이언트는 서버로부터 전달받은 쿠키를 활용
- 이들은 각각 응답 메시지의 Set-Cookie 헤더와 요청 메시지의 Cookie 헤더를 통해 전달
- 응답 메시지의 Set-Cookie 헤더
 - 쿠키의 이름, 값과 더불어 세미콜론(;)으로 구분되는 속성(들)을 전달
 - 한 응답 메시지에 전달할 쿠키가 여러 개라면 다음과 같이 여러 개의 Set-Cookie를 사용

응답 메시지

Set-Cookie: 이름=값

Set-Cookie: 이름=값; 속성1

Set-Cookie: 이름=값; 속성1; 속성2

HTTP 기반 기술

- 요청 메시지의 Cookie 헤더
 - 서버에 전달할 쿠키의 이름과 값을 나타내는 헤더
 - 여러 개의 쿠키 값을 서버에 전달해야 할 때는 세미 콜론(;)을 사용

요청 메시지

Cookie: 이름=값; 이름=값;

HTTP 기반 기술

- name="minchul", phone="100-100", message="Hello"라는 쿠키를 클라이언트에게 전송하는 서버의 응답 메시지

응답 메시지

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: name=minchul
Set-Cookie: phone=100-100
Set-Cookie: message=Hello
...헤더 후략...

...메시지 본문 생략...
```

- 서버로부터 전달받은 쿠키를 활용하는 클라이언트의 요청 메시지

HTTP 기반 기술

- 앞의 서버로부터 전달받은 쿠키를 활용하는 클라이언트의 요청 메시지

요청 메시지

```
GET /next_page HTTP/1.1
```

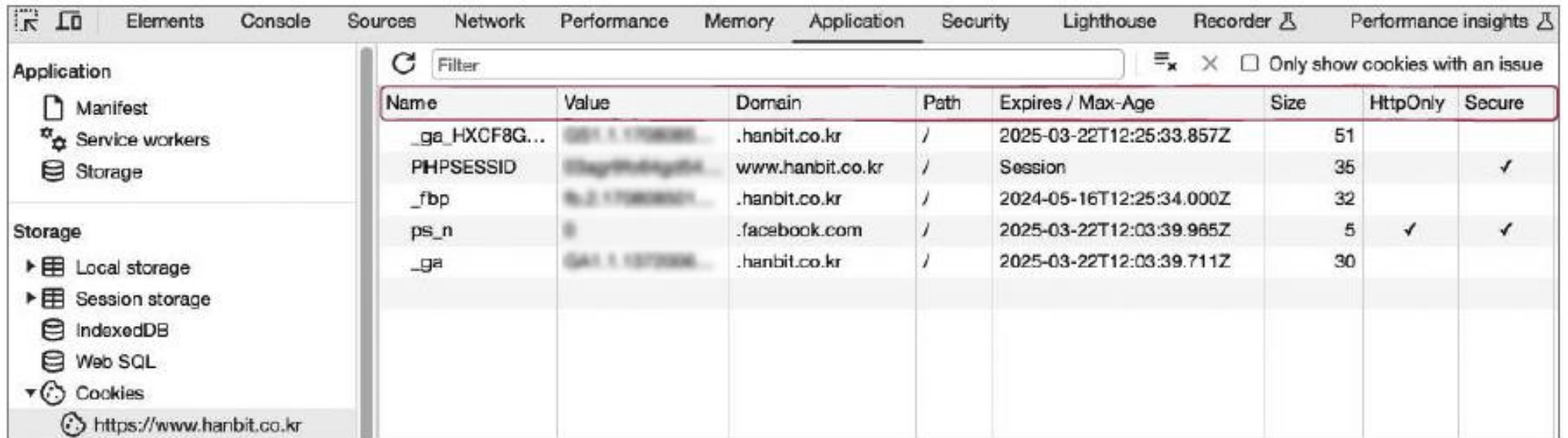
```
Host: example.com
```

```
Cookie: name=minchul; phone=100-100; message=Hello
```

```
...헤더 후락...
```

HTTP 기반 기술

- 크롬에서 쿠키 확인
 - 쿠키는 브라우저에서 저장되고 관리
 - 크롬 브라우저에서 개발자 도구[F12]를 열고, [Application] → [Storage] → [Cookies]를 확인



The screenshot shows the Chrome DevTools Application tab with the 'Storage' section expanded. The 'Cookies' subsection is selected, displaying a table of cookies for the URL https://www.hanbit.co.kr. The table includes columns for Name, Value, Domain, Path, Expires / Max-Age, Size, HttpOnly, and Secure. The cookies listed are: _ga_HXCF8G... (51 bytes, expires 2025-03-22T12:25:33.857Z), PHPSESSID (35 bytes, session, secure), _fbp (32 bytes, expires 2024-05-16T12:25:34.000Z), ps_n (5 bytes, expires 2025-03-22T12:03:39.965Z, httpOnly, secure), and _ga (30 bytes, expires 2025-03-22T12:03:39.711Z).

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure
_ga_HXCF8G...	GA1.1.1708888...	.hanbit.co.kr	/	2025-03-22T12:25:33.857Z	51		
PHPSESSID	02ag7h4h4g4h4...	www.hanbit.co.kr	/	Session	35		✓
_fbp	fb.1.1708888...	.hanbit.co.kr	/	2024-05-16T12:25:34.000Z	32		
ps_n	0	.facebook.com	/	2025-03-22T12:03:39.965Z	5	✓	✓
_ga	GA1.1.170888...	.hanbit.co.kr	/	2025-03-22T12:03:39.711Z	30		

HTTP 기반 기술

- 쿠키는 사용 가능한 도메인이 정해져 있음
 - 쿠키 관련 정보로 이름(Name)과 값(Value) 외에도 도메인(Domain)과 경로(Path) 등 `www.naver.com`에게 받은 쿠키를 전혀 다른 웹 사이트인 `www.google.com`에게 전송하면 안 됨
 - 응답 메시지 속 Set-Cookie 헤더의 'domain' 속성으로 정함

응답 메시지

```
Set-Cookie: name=minchul domain=example.com
```

HTTP 기반 기술

- 같은 도메인이라도 경로별로 쿠키를 구분하여 사용하고 싶을 경우
 - 예) `www.example.com/lectures`를 포함한 하위 경로에서 사용하고자 하는 쿠키와 `www.example.com/books`를 포함한 하위 경로에서 사용하고자 하는 쿠키가 다를 수 있음
 - 이럴 때는 다음 예시에서 붉은색 글자로 표시한 부분처럼 “path”로 쿠키가 적용될 경로를 명시
 - path로 지정된 경로와 그 앞부분이 일치하는 경로(하위 경로)에서 해당 쿠키 정보를 활용

응답 메시지

```
Set-Cookie: name=minchul path=/lectures
```

HTTP 기반 기술

- 쿠키의 유효 기간

- Expires는 [요일, DD-MM-YY HH:MM:SS GMT] 형식으로 표기되는 쿠키 만료 시점
- Max-Age는 초 단위 유효 기간
- Expires로 명시된 시점이 지나거나 Max-Age로 명시된 유효 기간이 지나면 해당 쿠키는 삭제되어 전달되지 않음

응답 메시지

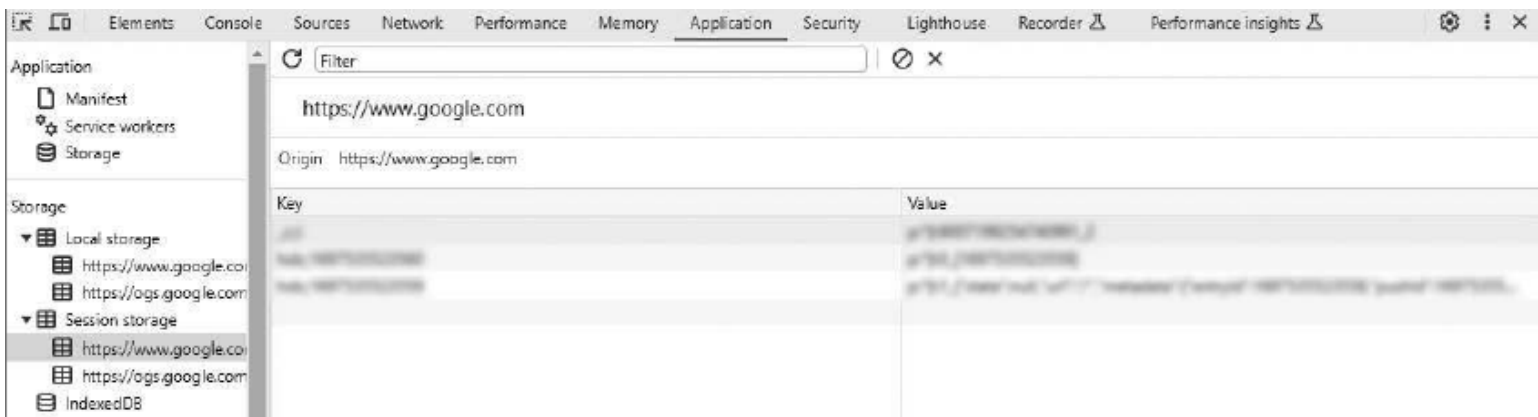
```
Set-Cookie: sessionId=abc123; Expires=Fri, 23 Aug 2024 09:00:00 GMT
```

```
Set-Cookie: sessionId=abc123; Max-Age=2592000
```

HTTP 기반 기술

- 쿠키의 한계
 - 쿠키의 대표적인 한계는 보안
 - 쿠키 정보는 쉽게 노출되거나 조작될 수 있음
- Secure와 HttpOnly
 - Secure는 HTTPS 프로토콜이 사용되는 경우에만 쿠키를 전송되도록 하는 속성
 - HTTPS 프로토콜은 HTTP를 더 안전한 방식으로 전송할 수 있는 프로토콜
 - HttpOnly는 HTTP 송수신을 통해서만 쿠키를 이용하도록 제한하는 속성
 - 쿠키 관련 데이터는 자바스크립트 언어를 통해서도 접근 가능
 - HttpOnly는 자바스크립트에서 쿠키에 접근하지 못하도록 하는 속성

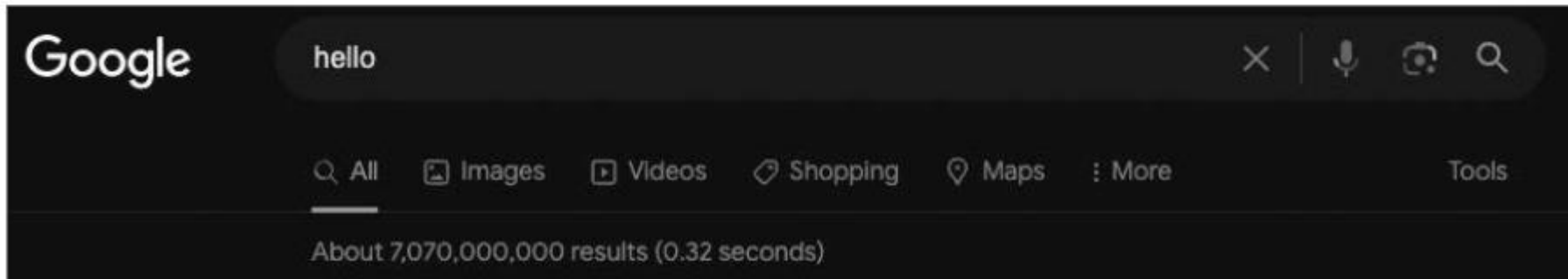
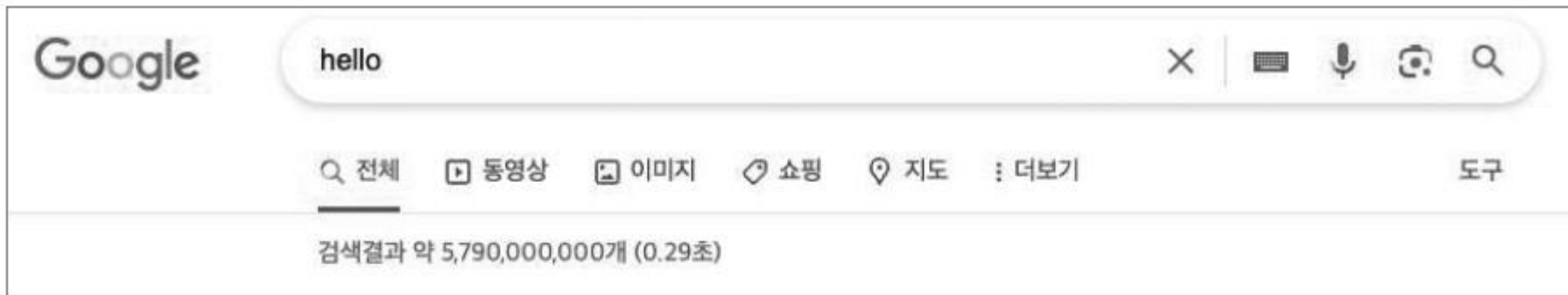
- **웹 스토리지(web storage)**
 - 클라이언트가 저장하고 클라이언트의 상태를 추측할 수 있는 <키-값> 쌍 형태의 정보
 - 웹 스토리지는 웹 브라우저 내의 저장 공간으로, 일반적으로 쿠키보다 더 큰 데이터를 저장
 - 쿠키는 서버로 자동 전송되지만, 웹 스토리지의 정보는 서버로 자동 전송되지 않고. 필요할 때 조회
- **웹 스토리지 종류**
 - 로컬 스토리지(local storage) - 별도로 삭제하지 않는 한 영구적으로 저장이 가능한 정보
 - 세션 스토리지(session storage) - 세션이 유지되는 동안(브라우저가 열려 있는 동안) 유지되는 정보
 - 개발자 도구를 열고 [Application] → [Storage]를 보면 로컬 스토리지와 세션 스토리지를 확인



HTTP 기반 기술

콘텐츠 협상과 표현

- 한국에서 접속하거나 한국어 계정으로 특정 URL에 접속하면 한국어로 된 웹 페이지를 볼 수 있고,
- 다른 지역에서 접속하거나 영어 계정으로 같은 URL에 접속하면 영어로 된 웹 페이지를 볼 수 있는 상황



- HTTP의 콘텐츠 협상(content negotiation)

- 같은 URI에 대해 가장 적합한 '자원의 형태'를 제공하는 메커니즘
 - 같은 URI로 식별 가능한 HTML 문서라 해도, 영어로 요청하면 영어로 된 형태로 제공하고, 한국어로 요청하면 한국어로 된 형태를 제공
- 자원의 표현(representation)
 - 송수신 가능한 자원의 형태
- 즉, 콘텐츠 협상은 클라이언트에게 가장 적합한 자원의 표현을 제공하는 메커니즘
- GET 메서드의 정의
 - '자원의 특정 표현을 습득하기 위한 메서드'

requests transfer of a current selected representation for the target resource.

↖ 대상 자원에 대해 현재 선택된 표현의 전송을 요청합니다.

◦ 콘텐츠 협상 관련 HTTP 헤더

- Accept 헤더
 - 선호하는 미디어 타입
- Accept-Language
 - 선호하는 언어
- Accept-Charset 및 Accept-Encoding
 - 선호하는 문자 인코딩과 압축 방식
- 예) 클라이언트가 선호하는 언어가 한국어 및 클라이언트가 HTML 문서 타입을 선호

요청 메시지

```
GET /index.html HTTP/1.1
Host: example.com
Accept-Language: ko
Accept: text/html
```

- 선호도에 우선순위를 반영

- 예) ‘언어는 한국어를 가장 선호하지만, 영어도 받을 용의가 있다’라는 식
 - 혹은 ‘미디어 타입은 HTML 문서를 가장 선호하지만, XML을 그 다음으로 선호하고, 일반 텍스트를 그다음으로 선호한다’라는 식
- 우선순위는 콘텐츠 협상 관련 헤더의 q 값으로 표현(q는 Quality Value의 약자)
 - 생략되었을 경우에는 1을 의미하고, 범위는 0부터 1까지이며, 값이 클수록 우선순위가 높음
- 예) 한국어(ko-KR, ko), 영어(en-US, en)순으로 선호하고, HTML, XML, 일반 텍스트순으로 선호

요청 메시지

```
GET /index.html HTTP/1.1
```

```
Host: example.com
```

```
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
```

```
Accept: text/html,application/xml;q=0.9,text/plain;q=0.6,*/*;q=0.5
```